

LeetCode Problem Pattern Recognition Framework

Goal

When you read a new problem, answer: 'Which algorithmic pattern should I use?'

Step 0 (Added Recommendation)

Estimate the maximum time complexity allowed from the constraints before thinking about algorithms. Eliminate impossible approaches first.

Step 1: Check Constraints

$n \leq 15$: Brute Force / Backtracking.

$n \leq 1000$: $O(n^2)$ may be acceptable.

$n \leq 100000$: Prefer $O(n)$ or $O(n \log n)$. Think Hash Map, Two Pointers, Sliding Window, Greedy, Binary Search, DP.

$n \geq 10^9$: Usually $O(\log n)$ or $O(1)$ only.

Step 2: Analyze Input

Tree → DFS/BFS.

Graph → DFS/BFS/Union Find.

Sorted Array → Binary Search or Two Pointers.

Linked List → Fast & Slow Pointers, Reversal, Merge.

Step 3: Examine Output

List of Lists → Backtracking.

Single optimized value → DP or Greedy (common hint, not a rule).

Modified Array/String → Two Pointers or In-place techniques.

Step 4: Scan Keywords

number of ways → DP

longest subsequence → DP

maximum/minimum → DP or Greedy

top k / k largest → Heap

parentheses → Stack

next greater element → Monotonic Stack

substring/window → Sliding Window

connected → Graph/Union Find

shortest path → BFS/Dijkstra

Decision Checklist

1. What complexity fits?
2. What data structure is given?
3. What output is expected?
4. Any keywords?
5. Start with the simplest valid approach.

Practice Tips

- Solve 5-10 problems from one pattern before switching.
- Guess the pattern before coding.
- Maintain your own keyword-to-pattern cheat sheet.
- After solving, ask why this pattern worked and why others did not.